TECHNICAL DEBT MANAGEMENT

TECHNICAL DEBT IS A BUSINESS RISK

Debt isn't just slow code — it is slow teams, brittle deploys, burnt-out engineers, and features that take weeks instead of days. Every sprint you ignore it, the interest compounds. Track it explicitly, prioritise it against features, and pay it down deliberately.

KEY BENEFITS

- ✓ Prevents the velocity cliff where adding features becomes nearly impossible
- ✓ Reduces engineer burnout caused by constant firefighting of fragile systems
- ✓ Shortens delivery lead time — clean code ships faster and with fewer defects
- ✓ Lowers incident rate as coupling and complexity are systematically reduced

REAL-WORLD EXAMPLE

Stripe intentionally allocates 30% of engineering time to debt reduction. Teams that don't do this create a self-reinforcing spiral: features become harder to ship, engineers burn out, and eventually the system stops evolving.

PRACTICAL EXAMPLE

Each sprint, the team's board has a fixed 'debt lane' — 20% of capacity. This week: extract a tangled billing module into a clean service. Next sprint, adding the new pricing model takes two days instead of two weeks.

REFLECTION QUESTION

How do you track technical debt? Is it on the backlog with business justification, or invisible until it causes a crisis?